



İOGYS

Teknik Dokümantasyon

İO Teknoloji

Sürüm 1.0 — Nisan 2026

Gizlilik: Şirket İçi

iogys.com.tr

İçindekiler

01 Sistem Mimarisi

02 Veritabanı Şeması

03 API Route Yapısı

04 Cron Job Sistemi

05 Arşiv Sistemi

06 Simülasyon Motoru

07 Yetkilendirme (RBAC)

08 Güvenlik

09 Deployment

10 Entegrasyonlar

BÖLÜM 01

Sistem Mimarisi

Genel Bakış

İOGYS (İO Görev Yönetim Sistemi), saha operasyonlarının dijital yönetimi için tasarlanmış kapsamlı bir web platformudur. Sistem, modern web teknolojilerinin sunduğu performans ve güvenilirlik avantajlarını bir araya getiren katmanlı bir mimari üzerine inşa edilmiştir. Kullanıcılar responsive bir web arayüzü üzerinden sisteme erişirken, arka planda güçlü bir sunucu altyapısı görev yönetimi, mesai takibi, çeklist denetimi ve raporlama gibi kritik iş süreçlerini yürütmektedir.

Platform, monolitik ancak modüler bir yapıda geliştirilmiştir. Next.js framework'ünün App Router mimarisi sayesinde her sayfa ve API endpoint'i bağımsız bir modül olarak çalışır. Bu yaklaşım, ekip içi geliştirme süreçlerini hızlandırırken aynı zamanda bakım maliyetlerini de düşük tutmaktadır. Server Components ile istemciye gönderilen JavaScript miktarı minimize edilmiş, sayfa yüklenme süreleri kritik düzeyde iyileştirilmiştir.

Frontend Katmanı

Ön yüz katmanı Next.js 14+ sürümü ile App Router paradigması kullanılarak geliştirilmiştir. TypeScript dili, tüm bileşen ve yardımcı fonksiyonlarda zorunlu olarak kullanılmakta; bu sayede derleme zamanında tip güvenliği sağlanmaktadır. React 18'in Concurrent Features desteği sayesinde kullanıcı arayüzü kesintisiz bir deneyim sunmaktadır. Tailwind CSS ile utility-first bir stil yaklaşımı benimsenmiş, bileşen kütüphanesi olarak özelleştirilmiş UI elementleri tercih edilmiştir.

Sayfa yönlendirmeleri App Router'ın dosya tabanlı route sistemi ile yönetilmektedir. Her tenant, firma ve proje düzeyinde dinamik segmentler kullanılarak izole edilmiş görünümde elde edilir. Layout dosyaları aracılığıyla ortak başlık, kenar çubuğu ve bildirim bileşenleri paylaşılmaktadır. İstemci tarafında state yönetimi için React Context API ve yerel hook'lar kullanılırken, sunucu tarafında veri çekme işlemleri doğrudan Server Components içinde gerçekleştirilmektedir.

Backend Katmanı

Sunucu tarafı mantığı tamamen Next.js API Routes üzerinde çalışmaktadır. Her API endpoint'i `app/api/` dizini altında organize edilmiş route handler dosyalarından oluşur. Bu handler'lar HTTP metodlarına göre ayrılmış `GET`, `POST`, `PATCH`, `DELETE` fonksiyonları içerir. Veritabanı işlemleri veritabanı istemcisi aracılığıyla yürütülmekte, her istekte kullanıcı kimlik doğrulaması JWT token kontrolü ile sağlanmaktadır.

Turhost üzerinde çalışan özel `server.js` dosyası, Next.js uygulamasını bir HTTP sunucusu olarak başlatırken aynı zamanda node-cron tabanlı zamanlayıcı görevleri de initialize etmektedir. Bu hibrit yaklaşım, ayrı bir worker servisi gerektirmeden periyodik arşivleme, simülasyon ve bildirim gönderimi gibi arka plan işlerinin yürütülmesini mümkün kılmaktadır.

Veritabanı ve Kimlik Doğrulama

Veritabanı olarak PostgreSQL veritabanı (8 GB) kullanılmaktadır. platform, yalnızca bir veritabanı hizmeti değil, aynı zamanda kimlik doğrulama (Auth), gerçek zamanlı abonelikler (Realtime) ve dosya depolama (Storage) gibi tamamlayıcı servisler de sunmaktadır. Kimlik doğrulama sürecinde JWT tabanlı cookie-based SSR auth yaklaşımı benimsenmiştir; bu sayede hem sunucu tarafında hem de istemci tarafında tutarlı bir oturum yönetimi sağlanmaktadır.

Yapay Zeka Entegrasyonu

İO Asistan modülü, Anthropic Claude API üzerinden çalışan bir yapay zeka destekli yardımcıdır. Anthropic SDK kullanılarak entegre edilen bu modül, Server-Sent Events (SSE) protokolü ile streaming yanıtlar üretmektedir. Kullanıcılar, saha operasyonlarıyla ilgili sorularını doğal dilde sorabilir; sistem bağlamsal bilgi ile zenginleştirilmiş yanıtlar döndürür. Rate limiting mekanizması ile API kullanımı kontrol altında tutulmaktadır.

Mimari Özet: Next.js (App Router) + TypeScript + PostgreSQL + Turhost + node-cron + Claude API. Tek bir deployment unit içinde hem web uygulaması hem de arka plan servisleri çalışmaktadır.

BÖLÜM 02

Veritabanı Şeması

Çekirdek Tablolar

Sistemin temel veri modeli, çok kiracılı (multi-tenant) bir yapı üzerine kurulmuştur. **firmalar** tablosu en üst düzey organizasyon birimini temsil eder; her firma altında bir veya birden fazla **projeler** kaydı bulunur. Kullanıcı bilgileri JWT Auth ile senkronize edilen **users** tablosunda tutulur. Her kullanıcı bir firmaya ve isteğe bağlı olarak bir veya birden fazla projeye atanabilir. **lokasyonlar** tablosu fiziksel çalışma noktalarını, **lokasyon_gruplari** ise bu noktaların mantıksal gruplandırmasını sağlar.

Firmalar tablosu, kiracı bazlı konfigürasyon bilgilerini de barındırır: aktif modüller, plan bilgisi, logo URL'leri ve iletişim detayları gibi alanlar bu tabloda yer alır. Projeler tablosu ise firma altındaki bağımsız iş birimlerini ifade eder ve her proje kendi lokasyon, görev ve personel havuzuna sahip olabilir. Bu yapı, tek bir İOGYS kurulumunun onlarca farklı firma ve yüzlerce projeyi eş zamanlı olarak yönetmesine imkan tanır.

Görev Tabloları

Görev yönetimi iki ana tablo etrafında şekillenmektedir. **gorevler** tablosu spesifik (tek seferlik veya belirli tarihe atanmış) görevleri içerirken, **canli_gorevler** tablosu frekansiyel olarak tekrarlanan görevleri barındırır. Her iki tablo da lokasyon, personel, proje ve durum bilgilerini foreign key ilişkileriyle tutar. **gorev_kurallari** tablosu ise frekansiyel görevlerin tekrar kurallarını (günlük, haftalık, aylık vb.) tanımlar ve bu kurallar doğrultusunda görevlerin otomatik olarak oluşturulmasını sağlar.

Görev durumları bir yaşam döngüsü izler: oluşturuldu, atandı, başladı, tamamlandı, iptal edildi. Her durum değişikliği zaman damgası ile kaydedilir. Tamamlanan veya iptal edilen görevler belirli bir süre sonra otomatik olarak arşiv tablolarına taşınır. Görev tabloları, fotoğraf URL'leri, açıklama metinleri, QR/NFC etiket referansları ve GPS koordinatları gibi saha verilerini de depolayacak şekilde tasarlanmıştır.

Çeklist Tabloları

Çeklist sistemi, şablon-sonuç ayrımı ile tasarlanmıştır. **checklist_sablonlari** tablosu çeklist tanımlarını, **checklist_sablon_maddeleri** her şablonun maddelerini ve **checklist_madde_secenekleri** ise maddelere ait seçenekleri (evet/hayır, puan, çoktan seçmeli vb.) tutar. Saha personeli bir çeklist doldurduğunda, sonuçlar **checklist_sonuc_basliklari** ve **checklist_sonuc_maddeleri** tablolarına yazılır. Bu ayrım sayesinde şablonlar değiştirilse bile geçmiş sonuçlar korunur.

Çeklist şablonları hiyerarşik bir yapıda organize edilebilir; kategoriler ve alt kategoriler aracılığıyla yüzlerce madde mantıksal bölümlere ayrılır. Her madde tipi farklı bir veri toplama yöntemini destekler: metin girişi, sayısal değer, fotoğraf yükleme, tarih seçimi ve çoktan seçmeli sorular bunların başlıcalarıdır. Şablon maddeleri sıra numarası ile düzenlenir ve zorunluluk durumu ayrıca belirlenebilir.

Arşiv Tabloları

Veritabanı performansını korumak amacıyla altı adet arşiv tablosu tanımlanmıştır: `personel_mesai_kayitlari_arsiv`, `musteri_degerlendirmeleri_arsiv`, `gorevler_arsiv`, `canli_gorevler_arsiv`, `checklist_sonuc_basliklari_arsiv` ve `checklist_sonuc_maddeleri_arsiv`. Bu tablolar, kaynak tablolarla birebir aynı şemaya sahiptir ve cron job tarafından otomatik olarak doldurulur. Arşive taşınan veriler, raporlama arayüzlerinden hala erişilebilir durumdadır.

Diğer Tablolar

Tablo	Açıklama	Temel İlişki
<code>personel_mesai_kayitlari</code>	Giriş-çıkış saatleri, GPS konum	users, lokasyonlar
<code>musteri_degerlendirmeleri</code>	Müşteri memnuniyet puanları ve yorumları	firmalar, projeler
<code>birim_fiyatlar</code>	Hizmet birim fiyat tanımları	firmalar
<code>bildirimler</code>	Uygulama içi bildirim kayıtları	users
<code>cron_log</code>	Cron job çalışma geçmişi	—
<code>device_tokens</code>	FCM push notification token'ları	users
<code>dashboard_bloklari</code>	Özelleştirilebilir dashboard widget konfigürasyonları	users, firmalar
<code>simulasyon_ayarlari</code>	Simülasyon motor parametreleri	firmalar, projeler
<code>simulasyon_personeller</code>	Simülasyon sanal personel havuzu	simulasyon_ayarlari

Tüm tablolar arasında PostgreSQL foreign key constraint'leri tanımlıdır. Cascade delete kuralları, bir firma silindiğinde ilişkili tüm verilerin tutarlı şekilde temizlenmesini garanti eder. Tablolar üzerinde performans için uygun indeksler oluşturulmuş, özellikle tarih ve durum alanları üzerinde composite index'ler tanımlanmıştır.

BÖLÜM 03

API Route Yapısı

Genel Mimari

İOGYS API katmanı, Next.js App Router'ın `app/api/` dizini altındaki route handler'ları ile oluşturulmuştur. Her endpoint, HTTP metodlarına karşılık gelen ve dışa aktarılan asenkron fonksiyonlar içerir. API tasarımında RESTful prensipler uygulanmış, kaynak adları URL path'inde, filtreler query string'de ve veri gövdesi JSON formatında iletilmektedir. Tüm yanıtlar tutarlı bir `NextResponse.json()` formatında döndürülür.

Kimlik doğrulama, her API çağrısının başında standart bir pattern ile gerçekleştirilir: `createClient()` fonksiyonu ile platform istemcisi oluşturulur, ardından `veritabanı istemcisi.auth.getUser()` çağrısı ile JWT token doğrulanır. Dönen kullanıcı kimliği `users` tablosundan sorgulanarak rol, firma ve proje bilgileri elde edilir. Bu bilgiler doğrultusunda yetki kontrolleri uygulanır ve yalnızca izin verilen veriye erişim sağlanır.

Temel Endpoint Grupları

Route Grubu	Kapsam	Temel İşlemler
<code>/api/auth/*</code>	Kimlik doğrulama	Login, logout, kayıt, şifre sıfırlama
<code>/api/tasks/*</code>	Görev yönetimi	CRUD, durum güncelleme, atama
<code>/api/mesai/*</code>	Mesai takibi	Giriş/çıkış kayıt, arşiv okuma
<code>/api/arsiv/*</code>	Arşiv verileri	Arşiv sorgulama, geri alma
<code>/api/raporlar/*</code>	Raporlama	Frekansiyel, personel, müşteri, çeklist raporları
<code>/api/simulasyon/*</code>	Simülasyon motoru	Ayar yönetimi, başlatma/durdurma
<code>/api/io-asistan</code>	Yapay zeka asistan	SSE streaming, bağlamsal soru-cevap
<code>/api/scan/*</code>	QR/NFC tarama	Etiket doğrulama, görev başlatma
<code>/api/app/*</code>	Mobil uygulama	Bildirim, cihaz kaydı, durum senkronizasyonu

Streaming ve Gerçek Zamanlı İletişim

İO Asistan endpoint'i, standart JSON yanıt yerine Server-Sent Events (SSE) protokolünü kullanır. İstemci tarafında `EventSource` veya `fetch` ile açılan bağlantı üzerinden yapay zeka yanıtları anlık olarak token token iletilir.

Bu yaklaşım, kullanıcının uzun yanıtları beklemek zorunda kalmadan ilk çıktığı anda görmesini sağlar. SSE bağlantısı, yanıt tamamlandığında sunucu tarafından kapatılır.

Raporlama endpoint'leri büyük veri kümeleri ile çalışabilmektedir. Performans için veritabanı sorgularında sayfalama (pagination) ve tarih aralığı filtreleri zorunlu tutulmuştur. Arşiv sorguları, hem aktif hem de arşiv tablolarını birleştirerek kapsamlı sonuç kümeleri döndürebilir. Müşteri değerlendirme endpoint'i, arşivlendi toggle özelliği ile veriyi tablolar arasında transfer edebilir.

Hata Yönetimi

Tüm API handler'ları try-catch blokları ile sarılmıştır. Hata durumunda uygun HTTP durum kodu (400, 401, 403, 404, 500) ile birlikte Türkçe hata mesajı döndürülür. Kimlik doğrulama hataları 401, yetki hataları 403 ve doğrulama hataları 400 kodu ile işaretlenir. Sunucu tarafı hatalar loglama altyapısına kaydedilirken, hassas hata detayları istemciye iletilmez.

BÖLÜM 04

Cron Job Sistemi

Mimari Tasarım

İOGYS'nin arka plan görevleri, Turhost platformu üzerinde çalışan özel bir sunucu yapısı ile yönetilmektedir. Standart Next.js uygulamaları serverless ortamlarda çalıştığında cron job desteği sınırlı kalır; bu nedenle `server.js` dosyası ile özel bir Node.js HTTP sunucusu oluşturulmuştur. Bu sunucu, Next.js request handler'ını saran bir katman olarak çalışır ve uygulama başlatıldığında `initArsivCron()` fonksiyonunu çağırarak zamanlayıcı görevleri aktifleştirir.

Zamanlayıcı altyapısı olarak `node-cron` kütüphanesi tercih edilmiştir. Bu kütüphane, Unix cron formatında tanımlanan zamanlamalara göre JavaScript fonksiyonlarını periyodik olarak çalıştırır. Turhost'in container tabanlı deploy mekanizması sayesinde, her deploy sonrası sunucu yeniden başlatıldığında cron görevleri otomatik olarak yeniden kayıt edilir. Bu yapı, ayrı bir worker servisi veya harici bir zamanlayıcı gerektirmeden güvenilir periyodik iş yürütme imkanı sunar.

Zamanlayıcı Görevleri

Görev	Periyot	Açıklama
Arşivleme	Her 6 saat	24+ saat eski kayıtları arşiv tablolarına taşır
Maksimum süre kontrol	Her 5 dakika	Aşırı uzun açık mesai kayıtlarını tespit eder
Simülasyon	Her 1 dakika	Aktif simülasyon senaryolarını ilerletir
Rapor gönderme	Her 15 dakika	Zamanlanmış raporları e-posta ile gönderir
Personel takip	Her 5 dakika	Saha personeli konum ve durum kontrolü
Gece döngüsü	pg_cron (gece)	Veritabanı bakım ve temizlik işlemleri
Bekleyen işlem	22:00 TRT	Gün sonu bekleyen görev ve mesai kontrolü
Personel destek	Vardiya sonları	Vardiya kapanış bildirimleri ve hatırlatmalar

Çalışma Mekanizması

Her cron görevi çalıştığında öncelikle `cron_log` tablosuna bir başlangıç kaydı oluşturur. İş tamamlandığında veya hata oluştuğunda bu kayıt güncellenir. Bu loglama mekanizması, cron görevlerinin sağlık durumunun

izlenmesini ve sorun giderme süreçlerinin hızlandırılmasını sağlar. Ayrıca, aynı görevin eş zamanlı olarak birden fazla kez çalışmasını engellemek için kilit (lock) mekanizması uygulanmıştır.

Arşivleme görevi, en kritik cron işidir ve altı farklı tabloyu sırasıyla işler. Her tablo için önce arşiv koşulunu sağlayan kayıtlar `INSERT INTO ... SELECT` ile arşiv tablosuna kopyalanır, ardından kaynak tablodan silinir. Bu işlem tek bir veritabanı transaction'ı içinde gerçekleştirilir; böylece herhangi bir hata durumunda kısmi veri kaybı önlenir. Simülasyon görevi ise her dakika çalışarak aktif simülasyon senaryolarındaki sanal personellerin görev durumlarını günceller.

Hata Toleransı

Cron görevleri, Turhost container yeniden başlatmalarına dayanıklı olacak şekilde tasarlanmıştır. `server.js` dosyası her başlatıldığında tüm cron görevleri sıfırdan kayıt edilir; bu nedenle önceki oturumdan kalan state bilgisine ihtiyaç duyulmaz. Her görev kendi try-catch bloğuna sahiptir ve bir görevin başarısız olması diğerlerini etkilemez. Kritik hatalar `console.error` ile loglanır ve gerektiğinde bildirim mekanizması tetiklenir.

BÖLÜM 05

Arşiv Sistemi

Tasarım Felsefesi

Arşiv sistemi, İOGYS veritabanının uzun vadeli performansını korumak amacıyla tasarlanmıştır. Saha operasyonları yoğun veri üreten süreçlerdir: her personelin günlük mesai kaydı, her görevin durum geçmişi ve her çeklist dolumu tablolarında birikerek sorgu performansını düşürür. Arşiv mekanizması, belirlenen yaş eşiğini (varsayılan 24 saat) aşmış kayıtları otomatik olarak ayrı tablolara taşıyarak aktif tabloların boyutunu kontrol altında tutar.

Bu yaklaşım, "sıcak veri — soğuk veri" ayrımı prensibine dayanır. Güncel ve aktif olarak sorgulanan veriler ana tablolarda kalırken, geçmişe dönük raporlama ihtiyaçları arşiv tablolarından karşılanır. Kullanıcı deneyimi açısından bu ayrım tamamen şeffaftır; raporlama arayüzleri her iki tablo grubunu da sorgulayabilir ve sonuçları birleşik olarak sunabilir.

Arşiv Tabloları ve Taşıma Kuralları

Kaynak Tablo	Arşiv Tablo	Koşul
personel_mesai_kayitlari	personel_mesai_kayitlari_arsiv	24+ saat eski
musteri_degerlendirmeleri	musteri_degerlendirmeleri_arsiv	24+ saat eski
gorevler	gorevler_arsiv	IPTAL veya 24+ saat TAMAMLANDI
canli_gorevler	canli_gorevler_arsiv	24+ saat eski tamamlanmış
checklist_sonuc_basliklari	checklist_sonuc_basliklari_arsiv	24+ saat eski
checklist_sonuc_maddeleri	checklist_sonuc_maddeleri_arsiv	Başlık ile birlikte

PostgreSQL Fonksiyonları

Arşivleme işleminin kritik bölümleri PostgreSQL fonksiyonları olarak tanımlanmıştır. `gun_sonu_arsivle()` fonksiyonu, gün sonu toplu arşivleme işlemini tek bir veritabanı çağrısı ile gerçekleştirir. Bu fonksiyon, tüm tabloları sırasıyla işler ve her adımda etkilenen kayıt sayısını döndürür. Transaction yönetimi PostgreSQL seviyesinde yapıldığından, uygulama katmanından daha güvenilir bir atomik işlem garantisi sağlanır.

`get_table_sizes()` RPC fonksiyonu, tüm İOGYS tablolarının boyut bilgilerini döndürür. Bu fonksiyon, yönetici panelinden çağrılarak veritabanı kapasite izlemesi yapılmasını sağlar. Tablo boyutları, satır sayıları ve

indeks boyutları gibi metrikler düzenli olarak kontrol edilerek kapasite planlaması yapılır. Arşivleme işleminin etkinliği bu metrikler üzerinden değerlendirilir.

API Entegrasyonu

Arşiv verileri, özel API endpoint'leri aracılığıyla erişilebilir durumdadır. `/api/mesai/arsiv` endpoint'i arşivlenmiş mesai kayıtlarını, `/api/raporlar/musteri-degerlendirme?arsivlendi=true` ise arşivlenmiş müşteri değerlendirmelerini döndürür. Çeklist raporları için `cikti=arsiv` parametresi kullanılarak hem aktif hem arşiv tablolarından veri çekilebilir. PATCH metodu ile tekil kayıtların arşiv durumu değiştirilebilir; bu işlem veriyi tablolar arasında fiziksel olarak transfer eder.

BÖLÜM 06

Simülasyon Motoru

Amaç ve Kapsam

Simülasyon motoru, İOGYS platformunun demo ve test ortamlarında gerçekçi saha operasyonu verileri üretmek için geliştirilmiş bir alt sistemdir. Potansiyel müşterilere sistemin yeteneklerini göstermek, eğitim senaryoları oluşturmak ve yük testi yapmak amacıyla kullanılır. Motor, gerçek saha operasyonlarının doğal akışını taklit eden parametrik bir algoritmaya sahiptir ve yapılandırma ayarlarına göre farklı senaryolar üretebilir.

Simülasyon, gerçek veritabanı tablolarına veri yazarak çalışır; bu sayede raporlama, dashboard ve bildirim gibi tüm sistem bileşenleri simülasyon verileriyle de sorunsuz çalışır. Bu yaklaşım, ayrı bir "demo modu" geliştirme ihtiyacını ortadan kaldırır ve sistemin tüm modüllerinin entegre olarak test edilmesini sağlar.

Veri Modeli

Simülasyon sistemi üç ana tablo üzerine kuruludur. `simulasyon_ayarlari` tablosu genel simülasyon parametrelerini tutar: aktiflik durumu, hedef oran, iptal olasılığı, erken ve geç sapma oranları gibi değerler bu tabloda tanımlanır. `simulasyon_grup_ayarlari` tablosu, lokasyon gruplarına özel parametreleri barındırır; bu sayede farklı bölgeler için farklı simülasyon profilleri oluşturulabilir. `simulasyon_personeller` tablosu ise simülasyonda kullanılacak sanal personel havuzunu içerir.

Her sanal personel, isim, cinsiyet, atanmış lokasyon ve vardiya bilgilerine sahiptir. Cinsiyet eşleştirmesi özelliği, gerçek saha koşullarına uygun personel ataması yapılmasını sağlar. Personel havuzu yönetici tarafından önceden tanımlanır ve simülasyon motoru bu havuzdan rastgele seçimler yaparak görev atamaları gerçekleştirir.

Vardiya Bazlı Doğal Akış

Simülasyon motoru, vardiya saatlerine göre doğal bir iş akışı simüle eder. Vardiya başlangıcında personel giriş kaydı oluşturulur, ardından belirlenen aralıklarla görev başlatma ve tamamlama olayları üretilir. Erken ve geç sapma oranları, görev tamamlanma sürelerindeki doğal varyasyonu taklit eder. İptal olasılığı parametresi, gerçek hayatta karşılaşılan görev iptali senaryolarını yansıtır.

Hedef oran parametresi, belirlenen süre içinde tamamlanması gereken görev yüzdesini ifade eder. Motor, bu orana ulaşmak için görev atama hızını dinamik olarak ayarlar. Vardiya sonunda tüm açık görevler tamamlanır veya ertesi güne aktarılır, personel çıkış kaydı oluşturulur. Bu döngü, cron job tarafından her dakika tetiklenerek sürekli bir veri akışı sağlar.

Konfigürasyon Parametreleri

Simülasyon parametreleri yönetici panelinden gerçek zamanlı olarak değiştirilebilir. Aktivlik toggle'ı ile simülasyon anında başlatılıp durdurulabilir. Oran parametreleri yüzde cinsinden girilir ve motor tarafından olasılık hesaplamalarında kullanılır. Grup bazlı ayarlar, belirli lokasyon gruplarında daha yoğun veya daha seyrek aktivite simüle edilmesine olanak tanır. Tüm parametre değişiklikleri bir sonraki cron çalışmasında otomatik olarak etkisini gösterir.

BÖLÜM 07

Yetkilendirme (RBAC)

Rol Hiyerarşisi

İOGYS, Rol Tabanlı Erişim Kontrolü (RBAC) modeli ile kapsamlı bir yetkilendirme sistemi sunar. Sistemde beş temel rol tanımlanmıştır ve her rol farklı erişim seviyelerine sahiptir. Roller hiyerarşik bir yapıda organize edilmiştir; üst roller, alt rollerin tüm yetkilerini miras alır ve ek yetkiler ekler.

Rol	Kapsam	Erişim Seviyesi
<code>super_admin</code>	Tüm sistem	Tüm firmalar, tüm veriler, sistem yönetimi
<code>alt_super_admin</code>	Tüm sistem (kısıtlı)	Firma yönetimi hariç tam erişim
<code>tenant_admin</code>	Kendi firması	Firma içi tüm projeler ve veriler
<code>tenant_user</code>	Atanmış projeler	Yalnızca yetkili projeler ve lokasyonlar
<code>musteri</code>	Atanmış lokasyonlar	Yalnızca değerlendirme ve görüntüleme

Sayfa Bazlı Yetkilendirme

Her kullanıcı rolü, sayfa bazında dört temel izin türüne sahip olabilir: `gorebilir` (görüntüleme), `ekleyebilir` (oluşturma), `duzenleyebilir` (güncelleme) ve `silebilir` (silme). Bu izinler, her sayfa için ayrı ayrı yapılandırılabilir. Örneğin, bir `tenant_user` görevleri görebilir ve yeni görev ekleyebilir ancak mevcut görevleri silemeyebilir. Bu granüler yapı, iş süreçlerine uygun esnek yetki konfigürasyonları oluşturulmasına olanak tanır.

Yetki kontrolleri hem sunucu hem de istemci tarafında uygulanır. Sunucu tarafında API handler'ları kullanıcı rolünü kontrol ederek yetkisiz istekleri reddeder. İstemci tarafında ise UI bileşenleri kullanıcının yetkilerine göre koşullu olarak render edilir: yetkisi olmayan butonlar gizlenir, erişilemeyen sayfalar için yönlendirme yapılır. Bu çift katmanlı yaklaşım, güvenliğin yalnızca arayüze bağlı kalmamasını garanti eder.

Lokasyon Bazlı Kısıtlama

`tenant_user` ve `musteri` rolleri için lokasyon bazlı erişim kısıtlaması uygulanmaktadır. Bu kullanıcılar yalnızca kendilerine atanmış lokasyonlara ait verileri görebilir ve işleyebilir. Lokasyon ataması `users` tablosundaki ilişki alanları üzerinden yönetilir. Grup yetkileri sayesinde bir kullanıcıya tek tek lokasyon atamak yerine bir lokasyon grubuna atama yapılabilir; gruba eklenen yeni lokasyonlar otomatik olarak kullanıcının erişim kapsamına girer.

Raporlama modülünde de aynı kısıtlamalar geçerlidir. Bir `tenant_user` yalnızca kendi lokasyonlarına ait mesai kayıtlarını, görev raporlarını ve çeklist sonuçlarını görüntüleyebilir. Bu kısıtlama veritabanı sorgusu seviyesinde uygulandığından, performans üzerinde minimum etki yaratır ve veri sızıntısı riski ortadan kalkar.

BÖLÜM 08

Güvenlik

Kimlik Doğrulama ve Oturum Yönetimi

İOGYS, JWT Auth altyapısı üzerine kurulu bir kimlik doğrulama sistemi kullanmaktadır. Kullanıcı girişi e-posta ve parola ile gerçekleştirilir; başarılı doğrulama sonucunda JWT (JSON Web Token) oluşturulur ve HTTP-only cookie olarak saklanır. Bu yaklaşım, XSS saldırılarına karşı token güvenliğini artırır. Token süresi dolduğunda refresh token mekanizması ile yeni token alınır; bu süreç kullanıcı tarafından fark edilmeden gerçekleştirilir.

Server-Side Rendering (SSR) ortamında cookie tabanlı kimlik doğrulama, hem sunucu hem de istemci tarafında tutarlı bir oturum deneyimi sağlar. Her sunucu tarafı sayfa isteğinde cookie'den JWT çıkartılarak kullanıcı kimliği doğrulanır. Geçersiz veya süresi dolmuş token tespit edildiğinde kullanıcı otomatik olarak giriş sayfasına yönlendirilir.

Veritabanı Güvenliği (RLS)

PostgreSQL üzerinde Row Level Security (RLS) politikaları etkinleştirilmiştir. RLS, veritabanı seviyesinde erişim kontrolü sağlayarak uygulama katmanındaki olası güvenlik açıklarının veri sızıntısına yol açmasını engeller. Her tablo için SELECT, INSERT, UPDATE ve DELETE operasyonlarına yönelik ayrı politikalar tanımlanmıştır. Bu politikalar, JWT token'ından çıkartılan kullanıcı kimliğine ve rolüne göre erişimi kısıtlar.

API Güvenliği

Cron job endpoint'leri `CRON_SECRET` environment variable'ı ile korunmaktadır. Harici bir kaynaktan cron endpoint'ine yapılan isteklerde bu secret header olarak gönderilmeli ve sunucu tarafında doğrulanmalıdır. Bu mekanizma, cron görevlerinin yalnızca yetkili kaynaklar tarafından tetiklenmesini garanti eder.

İO Asistan (Claude API) endpoint'i için rate limiting uygulanmıştır. Her kullanıcı belirli bir zaman diliminde sınırlı sayıda istek gönderebilir; bu sınır aşıldığında 429 (Too Many Requests) yanıtı döndürülür. Rate limiting, hem API maliyetlerini kontrol altında tutmak hem de olası kötüye kullanımı engellemek amacıyla tasarlanmıştır.

Ek Güvenlik Mekanizmaları

Tenant Admin seviyesindeki arşiv silme işlemleri için ek bir parola kapısı (password gate) tanımlanmıştır. Bu kritik işlem gerçekleştirilmeden önce kullanıcıdan mevcut parolasını tekrar girmesi istenir; parola JWT Auth üzerinden doğrulandıktan sonra işleme izin verilir. Bu yaklaşım, yanlışlıkla veya yetkisiz arşiv silme riskini minimize eder.

Tüm hassas environment variable'ları (veritabanı bağlantı bilgileri, API anahtarları, CRON_SECRET vb.) Turhost platform seviyesinde şifrelenmiş olarak saklanmaktadır. Kaynak kodda hiçbir hassas bilgi bulunmaz;

`.env.local` dosyası `.gitignore` ile versiyon kontrolünden hariç tutulmuştur. HTTPS zorunluluğu Turhost platformu tarafından otomatik olarak sağlanır ve tüm iletişim TLS 1.3 ile şifrelenir.

BÖLÜM 09

Deployment

Turhost Platform Yapısı

İOGYS, Turhost bulut platformu üzerinde deploy edilmektedir. Turhost, GitHub reposu ile entegre çalışan container tabanlı bir PaaS (Platform as a Service) hizmetidir. Her git push işlemi otomatik olarak yeni bir deployment tetikler: kaynak kodu Nixpack ile derlenir, Docker imajı oluşturulur ve yeni container başlatılır. Zero-downtime deployment sayesinde geçiş süreci kullanıcıları etkilemez.

Turhost'in tercih edilmesinin temel sebebi, daimi çalışan bir Node.js process'ine ihtiyaç duyulmasıdır. Vercel gibi serverless platformlar, cron job'lar için uzun süre çalışan process'leri desteklemez ve SMTP port kısıtlamaları gibi ek sınırlamalar getirir. Turhost'in container bazlı mimarisi, hem web sunucusunun hem de cron zamanlayıcısının aynı process içinde sorunsuz çalışmasına olanak tanır.

Başlatma Süreci

```
# Build aşaması
npm run build          # next build - üretim derlemesi

# Çalıştırma aşaması
npm start              # node server.js
  → Next.js HTTP handler başlatılır
  → initArsivCron() çağrılır
  → node-cron görevleri kayıt edilir
  → Sunucu PORT ortam değişkeninden dinlemeye başlar
```

`server.js` dosyası, Node.js `http.createServer` ile bir HTTP sunucusu oluşturur ve Next.js'in request handler'ını bu sunucuya bağlar. Ardından cron initialization fonksiyonunu çağırarak arka plan görevlerini başlatır. Turhost, `PORT` environment variable'ını otomatik olarak sağlar ve sunucu bu portu dinleyerek gelen istekleri işler.

Environment Variables

Değişken	Açıklama	Kaynak
<code>DATABASE_URL</code>	platform proje URL'i	platform Dashboard
<code>DATABASE_ANON_KEY</code>	platform anonim anahtar	platform Dashboard
<code>DATABASE_SERVICE_KEY</code>	platform servis anahtarı (RLS bypass)	platform Dashboard

Değişken	Açıklama	Kaynak
CRON_SECRET	Cron endpoint güvenlik token'ı	Manuel
RESEND_API_KEY	Resend e-posta API anahtarı	Resend Dashboard
ANTHROPIC_API_KEY	Claude API anahtarı	Anthropic Console
STRIPE_SECRET_KEY	Stripe ödeme API anahtarı	Stripe Dashboard
PORT	HTTP sunucu portu	Turhost (otomatik)
PUBLIC_DOMAIN	Public domain adresi	Turhost (otomatik)

Deploy Kontrol Listesi

Üretim deployment'ı öncesinde şu adımlar takip edilmelidir: veritabanı migration'ları platform Dashboard veya CLI üzerinden çalıştırılmalı, environment variable'ları Turhost Dashboard'da doğrulanmalı, build sürecinin hatasız tamamlandığı kontrol edilmeli ve deployment sonrası cron görevlerinin aktif olduğu `cron_log` tablosundan teyit edilmelidir. Geri alma (rollback) gerektiğinde Turhost'in önceki deployment'a dönme özelliği kullanılabilir.

BÖLÜM 10

Entegrasyonlar

E-posta — Resend

İOGYS, e-posta gönderimi için Resend HTTP API'sini kullanmaktadır. Turhost platformundaki SMTP port kısıtlamalarını aşmak amacıyla geleneksel SMTP yerine REST API tabanlı bir entegrasyon tercih edilmiştir. Resend, `iogys.com.tr` domain'i üzerinden doğrulanmış e-postalar göndermeye olanak tanır. Gönderen adı "İO Teknoloji" olarak yapılandırılmıştır ve tüm sistem bildirimleri, rapor gönderimleri ve kullanıcı davetleri bu kanal üzerinden iletilir.

E-posta şablonları HTML formatında hazırlanmıştır ve İOGYS marka renklerini kullanır. Rapor e-postaları, zamanlanmış cron görevi tarafından otomatik olarak tetiklenir ve alıcı listesi proje bazında yapılandırılabilir. Gönderim durumu ve hata logları Resend Dashboard üzerinden izlenebilir. Mail ayarları yönetici panelinden düzenlenebilir durumdadır ve Resend API anahtarı environment variable olarak güvenli şekilde saklanmaktadır.

Push Bildirimler — Firebase Cloud Messaging (FCM)

Mobil ve web push bildirimleri Firebase Cloud Messaging altyapısı ile gönderilmektedir. Kullanıcıların cihazlarından alınan FCM token'ları `device_tokens` tablosunda saklanır. Bildirim gönderimi, görev ataması, mesai hatırlatması, çeklist tamamlanması gibi olaylar tarafından tetiklenir. Her bildirim, başlık, gövde metni ve tıklanınca yönlendirilecek URL bilgisi içerir.

FCM entegrasyonu sunucu tarafında Firebase Admin SDK ile gerçekleştirilir. Toplu bildirim gönderimi için batch API kullanılarak tek bir istekte birden fazla cihaza bildirim iletilebilir. Token geçerliliği kontrol edilir; geçersiz token'lar otomatik olarak veritabanından temizlenir. Bu self-cleaning mekanizma, zamanla biriken eski cihaz kayıtlarının veritabanını şişirmesini önler.

Ödeme Sistemi — Stripe

İOGYS platformunun SaaS abonelik yönetimi Stripe ödeme altyapısı ile gerçekleştirilmektedir. Firma kayıtları sırasında veya plan yükseltmelerinde Stripe Checkout oturumu oluşturulur ve kullanıcı güvenli ödeme sayfasına yönlendirilir. Ödeme başarılı olduğunda webhook aracılığıyla İOGYS bilgilendirilir ve firma planı otomatik olarak güncellenir. Abonelik yenileme, iptal ve fatura işlemleri Stripe tarafından yönetilir.

Stripe webhook endpoint'i, gelen olayların imza doğrulamasını yaparak sahte istekleri reddeder. Desteklenen olay türleri arasında `checkout.session.completed`, `invoice.paid`, `customer.subscription.updated` ve `customer.subscription.deleted` yer alır. Her olay, ilgili firma kaydı üzerinde uygun güncellemeyi tetikler.

QR ve NFC Donanım Entegrasyonu

İOGYS'nin saha operasyonları yönetimindeki temel farklılaştırıcı özelliği, fiziksel QR kodlar ve NFC etiketleri ile dijital iş süreçlerini birleştirmesidir. Her lokasyona özgü QR kodlar ve NFC etiketler üretilir; saha personeli bu etiketleri tarayarak mesai girişi yapar, görev başlatır veya çeklist doldurma sürecini tetikler. /api/scan/* endpoint'leri tarama verilerini doğrular ve ilgili iş akışını başlatır.

QR kodlar, lokasyon kimliğini ve doğrulama hash'ini içeren şifreli veri taşır. Tarama sırasında bu veri sunucuya gönderilir, hash doğrulanır ve lokasyon eşleştirilir. GPS koordinatları ile çapraz kontrol yapılarak personelin gerçekten ilgili lokasyonda bulunduğu teyit edilebilir. NFC etiketleri de benzer bir doğrulama sürecinden geçer ve her iki teknoloji paralel olarak desteklenmektedir. Bu donanım entegrasyonu, sahada güvenilir ve hızlı bir veri girişi deneyimi sunar.